COMPUTER PROGRAMMING IN C++

Assignment 1 Solutions: Variables, Arrays, Loops & Control Flow Statements

Roll Number: 202510606

Course: Data Structures and Algorithms / Computer Science & Networking

Submission to: kamatekatende@gmail.com

> **IMPLEMENTATION GUIDELINES SATISFIED**
>
> - All functional components are modularized into independent sub-routines using references for standard persistence where needed.
> - Standard naming notations are explicitly respected throughout the file structure, with zero commentary as requested.

Complete Monolithic Solution Source Code (main.cpp)

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

const int DEFAULT_PIN = 12345;
const int MAX_ATTEMPTS = 3;
const int MAX_TRANSACTIONS = 100;

bool authenticateUser();
void showMainMenu();
void handleCheckBalance(int balance);
void handleDeposit(int &balance, string transactionHistory[], int &transactionCount);
void handleWithdraw(int &balance, string transactionHistory[], int
&transactionCount);
void handleTransactionHistory(const string transactionHistory[], int
transactionCount);
void showPrimaryBookMenu();
void handleHistorySection();
void handleConceptsSection();
void printFileContent(const string &fileName);

int main() {
    int balance = 0;
    string transactionHistory[MAX_TRANSACTIONS];
    int transactionCount = 0;
    int systemChoice;
```

```
    do {
        cout << "
================================" << endl;
        cout << "   COMBINED ACADEMIC SYSTEM PLATFORM" << endl;
        cout << "================================" << endl;
        cout << " [1] Run BK ATM Simulation System (Question 2)" << endl;
        cout << " [2] Run Foundations & History of C++ (Question 3)" << endl;
        cout << " [3] Exit Entire Program" << endl;
        cout << "================================" << endl;
        cout << "Enter system choice: ";
        cin >> systemChoice;

        if (systemChoice == 1) {
            cout << " [36m";
            cout << "           =========" << endl;
            cout << "             BK ATM SYSTEM  " << endl;
            cout << "           =========" << endl;
            cout << " [0m" << endl;

            if (!authenticateUser()) {
                cout << " [31m
ATM BLOCKED. Please contact your branch. [0m" << endl;
                continue;
            }

            int choice;
            do {
                showMainMenu();
                cout << "
Enter your choice: ";
                cin >> choice;

                switch (choice) {
                    case 1:
                        handleCheckBalance(balance);
                        break;
                    case 2:
                        handleDeposit(balance, transactionHistory, transactionCount);
                        break;
                    case 3:
                        handleWithdraw(balance, transactionHistory,
transactionCount);
                        break;
                    case 4:
                        handleTransactionHistory(transactionHistory,
transactionCount);
                        break;
                    case 5:
                        cout << " [33m
Thank You For Using BK ATM [0m" << endl;
                        break;
```

```
                    default:
                        cout << " [31m
Invalid Choice. Please try again. [0m" << endl;
                }
            } while (choice != 5);

        } else if (systemChoice == 2) {
            int userSelection;
            do {
                showPrimaryBookMenu();
                cout << "Select a numerical section choice: ";
                cin >> userSelection;

                switch (userSelection) {
                    case 1:
                        handleHistorySection();
                        break;
                    case 2:
                        handleConceptsSection();
                        break;
                    case 3:
                        cout << "
Closing the interactive book system." << endl;
                        break;
                    default:
                        cout << "
 [31mInvalid entry choice. Try matching menu values (1-3). [0m
" << endl;
                }
            } while (userSelection != 3);
        }
    } while (systemChoice != 3);

    return 0;
}

bool authenticateUser() {
    int enteredPin;
    int attemptsLeft = MAX_ATTEMPTS;

    do {
        cout << "       Enter your PIN: ";
        cin >> enteredPin;
        attemptsLeft--;

        if (enteredPin == DEFAULT_PIN) {
            cout << " [32m
        Login Successful [0m" << endl;
            cout << " [35m      Welcome to BK ATM [0m" << endl;
            return true;
        } else {
```

```
            cout << " [31m
Wrong PIN [0m" << endl;
            if (attemptsLeft > 0) {
                cout << " [33mRemaining Attempts: " << attemptsLeft << " [0m
" << endl;
            }
        }
    } while (attemptsLeft > 0);

    return false;
}

void showMainMenu() {
    cout << "
 [34m======= MAIN MENU =========== [0m" << endl;
    cout << " [1] Check Balance" << endl;
    cout << " [2] Deposit Money" << endl;
    cout << " [3] Withdraw Money" << endl;
    cout << " [4] Transaction History" << endl;
    cout << " [5] Exit" << endl;
    cout << " [34m============================== [0m" << endl;
}

void handleCheckBalance(int balance) {
    cout << " [32m
Current Balance: " << balance << " FRW [0m" << endl;
}

void handleDeposit(int &balance, string transactionHistory[], int &transactionCount)
{
    int depositAmount;
    cout << "
Enter amount to deposit: ";
    cin >> depositAmount;

    if (depositAmount <= 0) {
        cout << " [31mInvalid amount. [0m" << endl;
        return;
    }

    balance += depositAmount;
    cout << " [32m
Deposit Successful [0m" << endl;
    cout << "New Balance: " << balance << " FRW" << endl;

    if (transactionCount < MAX_TRANSACTIONS) {
        transactionHistory[transactionCount] = "Deposited: " +
to_string(depositAmount) + " FRW";
        transactionCount++;
    }
}
```

```
void handleWithdraw(int &balance, string transactionHistory[], int &transactionCount)
{
    int withdrawAmount;
    cout << "
Enter amount to withdraw: ";
    cin >> withdrawAmount;

    if (withdrawAmount <= 0) {
        cout << " [31mInvalid amount. [0m" << endl;
        return;
    }

    if (withdrawAmount > balance) {
        cout << " [31m
Insufficient Funds [0m" << endl;
    } else {
        balance -= withdrawAmount;
        cout << " [32m
Withdrawal Successful [0m" << endl;
        cout << "Withdrawn Amount: " << withdrawAmount << " FRW" << endl;
        cout << "Remaining Balance: " << balance << " FRW" << endl;

        if (transactionCount < MAX_TRANSACTIONS) {
            transactionHistory[transactionCount] = "Withdrawn: " +
to_string(withdrawAmount) + " FRW";
            transactionCount++;
        }
    }
}

void handleTransactionHistory(const string transactionHistory[], int
transactionCount) {
    cout << "
 [36m====== TRANSACTION HISTORY ====== [0m" << endl;
    if (transactionCount == 0) {
        cout << " [31mNo Transactions Found [0m" << endl;
    } else {
        for (int i = 0; i < transactionCount; i++) {
            cout << (i + 1) << ". " << transactionHistory[i] << endl;
        }
    }
}

void showPrimaryBookMenu() {
    cout << "
================================" << endl;
    cout << "   BOOK SYSTEM: Foundations and History of C++" << endl;
    cout << "================================" << endl;
    cout << " [1] Read History and Evolution of C++" << endl;
    cout << " [2] Study Core Programming Concepts" << endl;
```

```
    cout << " [3] Close Program" << endl;
    cout << "==============================" << endl;
}

void handleHistorySection() {
    int subChoice;
    cout << "
---------------------------------------" << endl;
    cout << " CHAPTER SELECTION: History and Evolution" << endl;
    cout << "---------------------------------------" << endl;
    cout << " [1] Complete Language Tree (Fortran to C++std)" << endl;
    cout << " [2] Return to Main Menu" << endl;
    cout << "---------------------------------------" << endl;
    cout << "Enter sub-chapter choice: ";
    cin >> subChoice;

    if (subChoice == 1) {
        cout << "
--- LOADING HISTORY CHRONOLOGY FROM FILE ---" << endl;
        printFileContent("history.txt");
    }
}

void handleConceptsSection() {
    int subChoice;
    cout << "
---------------------------------------" << endl;
    cout << " CHAPTER SELECTION: Core C++ Concepts" << endl;
    cout << "---------------------------------------" << endl;
    cout << " [1] File Streams, Variables, Loops & Arrays Summary" << endl;
    cout << " [2] Return to Main Menu" << endl;
    cout << "---------------------------------------" << endl;
    cout << "Enter sub-chapter choice: ";
    cin >> subChoice;

    if (subChoice == 1) {
        cout << "
--- LOADING CONCEPT CODES FROM FILE ---" << endl;
        printFileContent("concepts.txt");
    }
}

void printFileContent(const string &fileName) {
    ifstream targetedFile(fileName);

    if (!targetedFile.is_open()) {
        cout << " [33m[Notice] File "" << fileName << "" was not found. [0m" << endl;
        cout << "Fallback Mode: Please create this file in your folder path to view
custom text." << endl;
        return;
    }
```

```
    string fileLine;
    cout << " [32m
================================" << endl;
    while (getline(targetedFile, fileLine)) {
        cout << fileLine << endl;
    }
    cout << "================================ [0m
" << endl;
    targetedFile.close();
}
```